Asynchronous Processing

move slow, unreliable work off the request path

Watch the 4-min video

In one line: the request handler does only what the user needs now and puts everything else on a queue for background workers; make slow, unreliable, retryable or batchable work asynchronous, report long tasks with 202 Accepted plus a status URL, and make tasks idempotent with retries, backoff, a dead-letter queue and monitoring.

1 Eight seconds to sign up

A user clicks sign up. Your API creates the account, sends a welcome email, resizes their profile photo into five sizes and syncs them to the CRM. Eight seconds later the page finally responds, if the email provider didn't time out first. Most of that work didn't need to happen while the user waited.

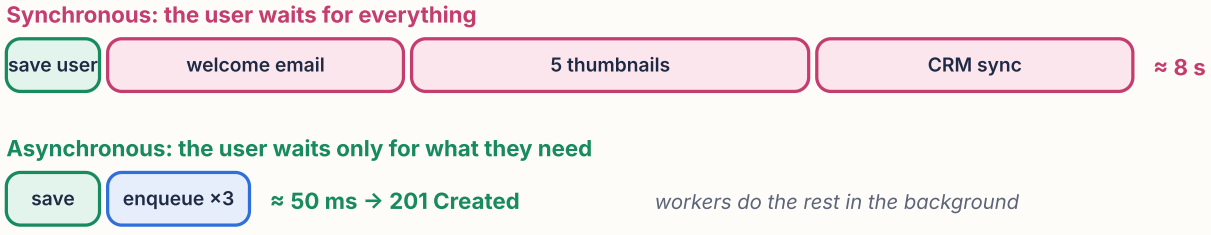


Figure 1. The same sign-up done synchronously and asynchronously.

2 The pattern

The request handler does only what the user needs **right now**: validate input, save the account. Everything else becomes a **task**, a message put onto a **queue**. Separate **worker** processes pull tasks from the queue and do the slow work in the background. The API responds in milliseconds, and a slow or failing email provider no longer breaks sign-up.

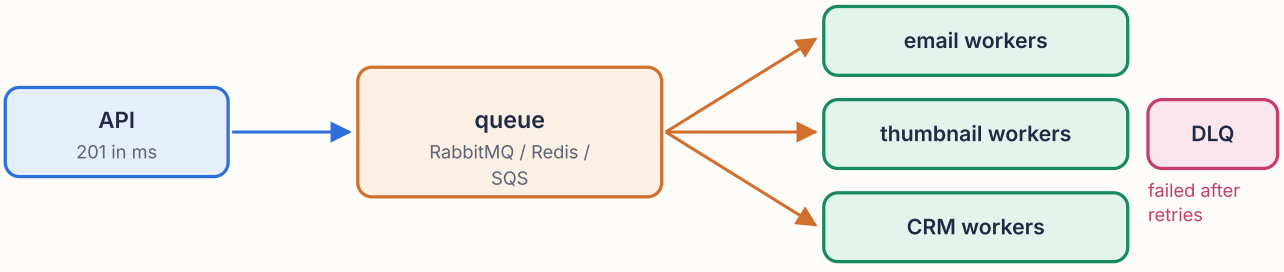


Figure 2. API, queue, workers: the queue absorbs bursts, workers scale independently, and repeatedly failing tasks go to a dead-letter queue.

3 What should be asynchronous?

- **Slow** work: video encoding, reports, AI model calls.
- **Unreliable** work: third-party APIs that can fail or time out.

- **Retryable** work: emails, webhooks, notifications.
- **Batchable** work: analytics, search indexing, bulk updates.

Keep work synchronous when the user needs the result immediately to continue: the payment authorisation at checkout, the login check.

4 Reporting status: 202 Accepted

For long tasks the user is waiting on, the API returns **202 Accepted** with a task ID and a status URL (Phase 2, API lesson 51). The client polls that URL, or is notified when the task finishes through a WebSocket, server-sent events, a push notification or an email. Task status lives in a database or Redis: *pending*, *running*, *succeeded* with a result link, or *failed* with a reason.

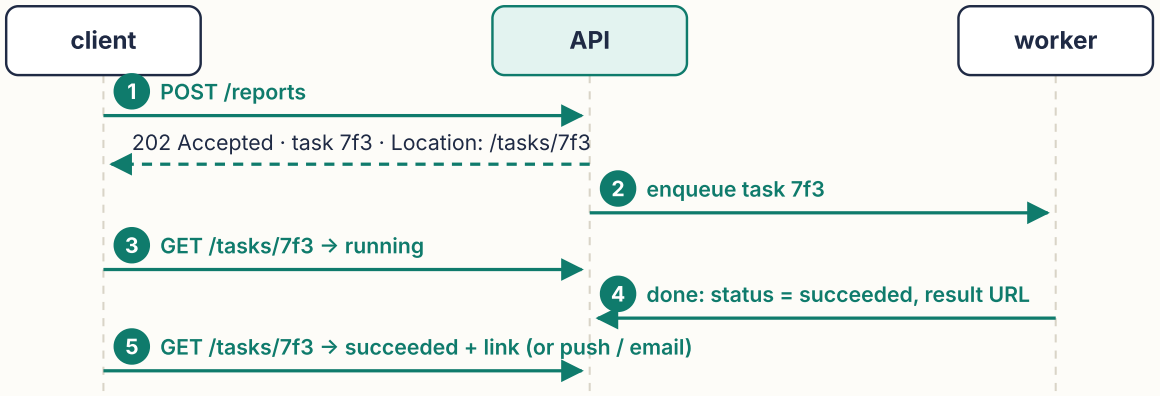


Figure 3. The 202 pattern: accept, return a task ID at once, and let the client poll or be notified.

5 Tools

In Python, **Celery** is the classic task queue (brokers RabbitMQ or Redis, retries, scheduling with beat, result backends). **RQ** is a simpler option built on Redis; **Arq** is designed for asyncio. In the cloud, managed queues like **SQS** or **Azure Service Bus** with serverless or container workers do the same job.

WORKED EXAMPLE: SIGN-UP, ASYNCHRONOUSLY

On sign-up the API saves the user, enqueues `send_welcome_email` and `generate_thumbnails`, and returns **201** in about 50 ms. Email workers call the provider with retries and exponential backoff (1 s, 2 s, 4 s ... up to an hour). Thumbnail workers resize the uploaded image into five sizes and store them in S3. The user sees their new account instantly, and thumbnails appear a few seconds later.

6 Background work needs care

- **Idempotent tasks**: a task may run more than once after a crash or retry (at-least-once delivery), so a second run must do no harm, e.g. check "email already sent for user 42".
- **Retries with backoff** and jitter, and a **dead-letter queue** for tasks that keep failing, so they don't block the queue and someone can inspect them.
- **Monitor** queue depth and the age of the oldest task, so you notice when workers fall behind.
- **Autoscale workers** on queue length (KEDA, lesson 14).
- Enqueue only **after** the database commit (or use a transactional outbox), or a worker may look for a user that doesn't exist yet.

COMMON MISTAKES

- Putting large payloads (the image itself) in the message instead of a reference (the S3 key).
- Non-idempotent tasks that send two welcome emails after a retry.
- No visibility into the queue, so a backlog of 2 million tasks is found by a customer.

7 Key terms

Task / job: a unit of background work, as a message

Worker: process that executes tasks

Dead-letter queue: where repeatedly failing tasks go

Transactional outbox: write the event in the same DB transaction, publish after

Queue / broker: holds tasks until a worker takes them

202 Accepted: request accepted, processing later

At-least-once: a task may run more than once

8 Interview questions

Q1. What work should be done asynchronously? BASIC

Anything slow, unreliable, retryable or batchable that the user doesn't need immediately, like emails, thumbnails, reports and third-party calls.

Q2. How do you report task status? BASIC

Return 202 Accepted with a task ID and a status URL, store status in a database or Redis, and let the client poll it or notify them through WebSockets, server-sent events, push or email.

Q3. Design PDF report generation that takes 2 minutes. SCENARIO

POST returns 202 with a task ID; a worker builds the PDF, stores it in S3 and updates the task to succeeded with a presigned link; the UI polls or gets an SSE event; tasks are idempotent and retried with backoff.

Q4. Workers fall behind every Monday morning. What do you do? SCENARIO

Autoscale workers on queue depth and oldest-task age, separate queues by priority so urgent tasks aren't stuck behind bulk ones, and batch where possible.

Q5. Why must background tasks be idempotent? TRICKY

Queues deliver at least once: a worker can crash after doing the work but before acknowledging, so the task runs again. Idempotency (checks, keys, upserts) makes repeats harmless.

Q6. A worker can't find the user it was asked to email. Why? TRICKY

The task was enqueued before the database transaction committed. Enqueue after commit, or use a transactional outbox so the event is published only once the data exists.

Next: SD-J-005 · Python Concurrency Models backendroadmap.com